

Lecture 20

Tuesday April 21st 2026

Final will cover everything after the midterm: Sync, deadlock, memory management

Memory Management

- The OS must fit multiple processes in memory
 - memory needs to be subdivided to accommodate multiple processes
 - Memory needs to be allocated to ensure a reasonable supply of ready processes so that the CPU is never idle
 - Memory management is an Optimization task under constraints

Memory Allocation

1. Fixed Partition allocation

- divide memory into fixed-size partitions
- each partition contains exactly one process
- the degree of multi programming is bound by the number of partitions
- when a process terminates, the partition becomes available for other processes
- may lead to internal fragmentation - allocated memory may be slightly larger than requested memory
- this size difference is memory internal to a partition, but not being used

2. Variable partition scheme (dynamic)

- When a process arrives, search for a hole large enough for the process
- Hole is block of available memory: various different sizes
- allocate only as much memory as needed
- Operating system maintains information about:
 - allocated partition
 - free partitions (hole)

Memory allocation cont.

- Can lead to external fragmentation - total memory space exists to satisfy a request, but it is not contiguous (in average ~ 50% lost)
- Reduce external fragmentation by compaction
 - shuffle memory contents to place all free memory together in one large block
 - compaction is possible only if relocation is dynamic, and is done at execution time

Dynamic Storage

How to satisfy a request of size n from a list of free holes

- **first fit:** allocate the first hole that is big enough
- **Best fit:** allocate the smallest hole that is big enough; must search entire list, unless ordered by size. Produces the smallest leftover hole
- **Worst fit:** allocate the largest hole; must also search entire list. Produces the largest leftover hole

first-fit: faster

best-fit: better in terms of storage utilization

worst-fit: may lead to less fragmentation

Address Binding

- Addresses in a source program are generally *symbolic* (`int count;`)
- A compiler **binds** these symbolic addresses to **relocatable** (logical) addresses
- The linkage editor or loader will in turn bind the relocatable addresses to **Absolute** (physical) addresses (`0xFFFFFFFF`)
- Each binding is **mapping** from one address space to another

Logical Address Space

- Each process has a separate memory space
- Two registers provide address protection between processes:
 - **Base register:** smallest legal address space
 - **Limit register:** size of the legal range

Memory- Management Unit MMU

- Hardware device that maps logical to physical address
- In MMU scheme, the value in the **relocation register** (base register) is added to every address generated by a user process at the time it is sent to memory
- The **user program** deals with *logical* addresses; **it never sees the real physical address**

HW Address Protection

- CPU hardware compares every address generated in user mode with the registers
- any attempt to access other processes' memory will be trapped and cause a **fatal error**

Paging - Non-Contiguous

- Physical address space of a process can be non-contiguous
- Divide physical memory into fixed-sized blocks called **frames** (size is a power of 2, between 512b and 16mb)
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size n pages, need to find n free frames and load program
- Set up a page table to translate logical to physical addresses

Paging Address Translation Scheme

- Address generated by CPU is divided into:
 - **Page number (p)** - used as an index into a *page table* which contains base address of each page in physical memory
 - **Page offset (d)** - combined with base address to define the physical memory address that is sent to memory unit

Shared Pages

- **Shared code**
 - One copy of read-only (re-entrant) code shared among processes (i.e text editors, compilers, window systems)
 - shared code must appear in same location in the logical address space of all processes
- **Private code and data**
 - Each process keeps a separate copy of the code and data
 - the pages for the private code and data can appear anywhere in the logical address space

Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments.
- A segment is a logical unit such as:
 - main program
 - procedure
 - function
 - method
 - object
 - local and global variables
 - common block
 - stack

- symbol table
- arrays